

Using Cloud Infrastructure as a Code IaC to Set Up Web Applications

Mohammed W. Eleraky

Department of Media Engineering and Technology

Faculty of Engineering

German University in Cairo

mohammed.eleraky@student.guc.edu.eg

Wagdy A. Aziz

Faculty of Engineering

Ain Shams University/Orange-Egypt

wagdy.anis@orange.com

John N. Soliman

M.Sc., Faculty of Engineering

Ain Shams University

john.n.soliman@ieee.org

Abstract—This paper intends to explore the capabilities of Infrastructure as a Code (IaC), and tackling some of the cloud of the resources it can create. This study assumes that a cloud developer has access to resources regarding IaC, as well as extensive knowledge on how to use them. The goal in this study is to create an enterprise-ready solution through the usage of IaC exclusively. IaC would be used to create a fully functional network infrastructure, including a network, subnets, and gateways; a website capable of handling heavy loads, through the usage of a load balancer, and recover in the case of disasters or failures, through the usage of an auto-scaling group; and a database that an enterprise may use; through the usage of the relational database service. Additional features that are offered by AWS's CloudFormation are also investigated. IaC was capable of achieving the goals set, with room for modifications for a more complex solution.

Index Terms—cloud, infrastructure as a code, IaC, cloud computing, AWS

I. INTRODUCTION

A. The Cloud

A while back, entrepreneurs would often be discouraged from setting up a website and network infrastructure for their businesses due to the cost and complexity of setting them up as well as maintaining them, which led to the rise of a technology known as the Cloud, allowing not only those entrepreneurs, but also established businesses to easily set up their website and network infrastructure.

Since its introduction in the commercial sector, the cloud has undergone a significant change in storing and securing information. Through the cloud, data are run in a collection of nodes including servers and remote computers, which enables users to remotely access the data at any time and from any location [1]. For consumers, the cloud is generally considered in the much simpler terms of storage, settings and software. This equates to a simple definition of the cloud as a place for storing and accessing data and programs over the internet instead of on a computer's hard drive [2]. The cloud service providers wish to ensure the delivery of flexible services offered in such a way that keeps users separated from the underlying infrastructure [1]; these services are defined under an umbrella term referred to as cloud computing.

Cloud computing is important when applied to data recovery due to its flexibility, cost-effectiveness, reliability, and scalabil-

ity [1]. One of the largest outward uses of cloud calculating is the mobility that it brings, both to the commercial and the business user. Many of us are already conversant with some Cloud Computing services, like Google Docs, or even email services. The utmost popular Cloud Computing products include AWS Elastic Compute, Google Cloud Engine and AWS Lambda [3]; additionally, cloud computing powers streaming services that we use on a daily basis, such as: content streaming services like Netflix and YouTube, social media platforms like WhatsApp and Twitter, and even gaming, as they may be used to either host or stream content to users. The most famous cloud computing services are including Amazon Web Services (AWS), Google Cloud platform (GPC) and Microsoft Azure; cloud computing provides a lot of flexibility, providing businesses with the ability to scale their infrastructure as needed; for instance, should their demands increase, it's easy to increase their cloud capacity, drawing on the service's remote servers [3].

B. Infrastructure As A Code (IaC)

IaC may be defined as the “source code” of the cloud infrastructure, and this term wasn't obtained without a good reason. The entire set of scripts, automation and configuration code, models, required dependencies and operational configuration parameters that may be expressed using the same standard language, much like you would use Java and ancillary Java libraries to put together your own software application, is IaC [4]. IaC allows developers to create templates from which they may create the same infrastructure as needed, without having to waste time dealing with interfaces and matching the specifications; this option is of critical importance to companies with multiple headquarters.

IaC is an approach for infrastructure automation that is based on software development practices [4]. IaC promotes the usage of code to define and configure the IT infrastructure of cloud applications, allowing them to benefit from conventional software development practices, which facilitates the rapid deployment of new versions of application infrastructures without sacrificing quality or stability. IaC also enables better compliance management of applications, as it allows for the uniform management of the compliance of the infrastructure of applications that utilize heterogeneous IaC technologies for

deployment automation [5]. Additionally, IaC can help prevent security and reliability issues in the deployed infrastructure by using static analysis techniques to detect and prevent defects in IaC scripts [6].

II. BACKGROUND

A. Cloud Service Models

To better understand the cloud and to familiarize you with the topic of the study, one should be familiar with the models by which its services are offered; while there are tens of models, the three most relevant ones to this study will be discussed to introduce the topic on which this study was conducted.

1) *Software-as-a-Service (SaaS)*: SaaS is a model that defines software hosted on the cloud; this can be defined as applications which the business's users may use in order to perform functionalities that the business offers, think of your social media and streaming apps; reliance on SaaS minimizes the support and maintenance costs effectively. Recent reports show that SaaS will soon become common in every organization and it is important that buyers and users of technology understand what SaaS is and where it is suitable [7].

2) *Infrastructure-as-a-Service (IaaS)*: IaaS is a model that defines the usage of the cloud in order to host the infrastructure, be it network, storage, or servers; instead of relying on physical hardware in order to have a functioning infrastructure, companies may instead rely on cloud providers in order to host the infrastructure instead. Internet Engineering Task-Force (IETF) has defined the IaaS model to be the most basic service model. This model is often described as a virtual engine [8].

3) *Database-as-a-Service (DBaaS)*: DBaaS is a model of cloud computing that allows users to have their own database, which they can control as they see fit, hosted on the cloud in its entirety; no longer would companies need to purchase terabytes and petabytes of storage in order to accommodate their needs, but would instead rely on Application Program Interface (API) to manage their database. This API is available to the user by management console – Web application, which the user may use to manage and configure the database and even provision or deprovision database instances [7].

B. IaC: Benefits

Using IaC provides a plethora of benefits; the following subsection will be providing insights into those benefits and how they may affect a business owner planning for the move towards IaC.

1) *Lower Costs*: Since significant tasks, like time-consuming infrastructure configuration, are automated by IaC, the engineers or IT experts can finish these tasks in no time and focus on other mission-critical tasks [9].

2) *Speed*: The automation of the creation process of infrastructure allows developers and enterprises to save a lot of time that could be used to get other tasks done.

3) *Low risk of human errors*: There are two descriptions of human errors. The first is the face meaning which implies mistakes made by engineers or IT personnel in the manual process of infrastructure building. The second side to human error is the adaptation of new employees to the infrastructure built by the former IT expert [9]; automation and documentation, due to IaC being code, allows for the extermination of the aforementioned problems that faced developers previously.

4) *Improved Consistency*: Since the process is automated, every time the code is uploaded, it will follow the same structure and process as it did previously; for instance, one developer may prefer writing the resources in an IaC file in a manner different from another developer, but with the usage of IaC, this is not an issue [9].

5) *Eliminate configuration drift*: Amongst the many advantages of this idempotency feature, the most striking is how IaC averts configuration drift. If someone alters a resource, not in sync with your IaC pipeline, you need to correct this and get the resources back to the correct state as quickly as possible, but with IaC, the specifications of your environment configuration are already in the code. This implies that the correction happens automatically if the change is implemented [9].

6) *Improved security strategies*: Since IaC does everything in the same manner, this allows developers to adhere to security regulations every time it the files are uploaded; on top of that, IaC offers more control over who may access and modify the code through authorization measures [9].

7) *DevOps*: Additionally, IaC plays a great role in when it comes to the development process; IaC is one of the key enablers of automation and streamlining [4]. For instance, IaC enables cloud developers to create solutions through the usage of scripts, allowing developers to save a lot of time that would have otherwise been wasted navigating graphical user interfaces (GUI) and ensuring that everything is working accordingly; IaC allows developers to save their solutions and architecture in the form of blueprints that may either be used as a baseline for other solution, or distributed and used by other developers for the same purpose as well.

III. METHODOLOGY

For this solution, it was decided that the creation of the infrastructure should be split into three phases: planning, implementation, and testing. During the planning phase, the functionalities of the infrastructure are to be determined: how should the network be, what services should be deployed, and how should they be deployed.

A. Planning Phase

The goal was the creation of a deploy-able infrastructure that may be utilized by enterprises immediately, so an enterprise was found to need a network infrastructure on which they may host their website and services while also being stable and accommodating for heavy loads and traffic, and they will also need a database that they may use to store their user and business data; additionally, they may need a website

to be up and running quickly. Once the services needed are determined, a developer needs to look for the most appropriate region on which they may host their infrastructure; this choice is mostly based on two things: the latency relative to the target demographic and the services needed for the infrastructure.

For the latency, please refer to the image below and follow the link: [10] to determine the best region that you may use.



Fig. 1. A map showing the different AWS regions.

Once a developer has an idea of which region is would offer the least latency, they would then need to check whether the region supports all of the services that they wish to incorporate into their infrastructure; for this study, the region EU-Central (Frankfurt) was found to fit perfectly according to our requirements.

With the region determined and the requirements known and understood, the tool available through [11] was used in order to sketch a diagram to act as guideline for our implementation; Lucid Chart allows developers to easily sketch a diagram through using shapes that are representative of AWS. The following diagram was the one that we settled on, and which will be used to explain the infrastructure implemented.

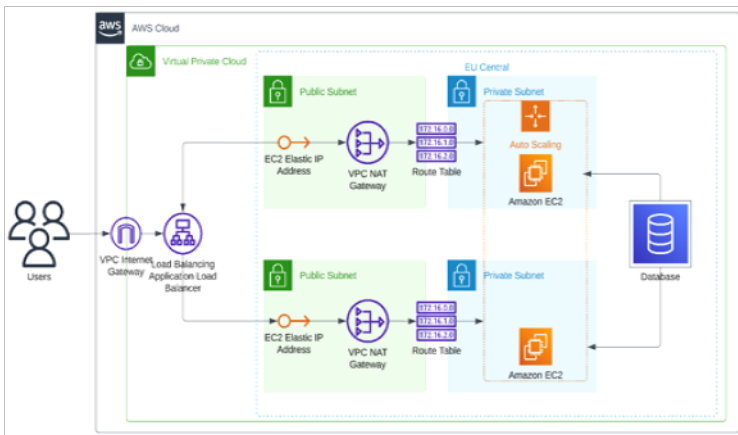


Fig. 2. A map showing the different AWS regions.

B. Implementation Phase: Parameter File

With our infrastructure determined, the next step would be the actual implementation and creation of the specified

resources. In order to facilitate the understanding of the infrastructure, the item will be ordered according to how crucial they are for the infrastructure.

Before we get into the actual creation of the resources, it is conventional to create a parameter file which will be used to hold the values for the variables that will be used in the infrastructure. Some variables, such as the name of the environment and the region, are better to be defined in their own file, which allows us to save time while creating the infrastructure files and maintain the infrastructure in an easier method. The parameter file is written in JSON, while the rest of the files are written in YAML.

```
[
  {
    "ParameterKey": "EnvironmentName",
    "ParameterValue": "HA-Project"
  },
  {
    "ParameterKey": "VpcNetCIDR",
    "ParameterValue": "10.0.0.0/16"
  },
  {
    "ParameterKey": "PubSubnet01CIDR",
    "ParameterValue": "10.0.1.0/24"
  },
  {
    "ParameterKey": "PubSubnet02CIDR",
    "ParameterValue": "10.0.2.0/24"
  },
  {
    "ParameterKey": "PrivSubnet01CIDR",
    "ParameterValue": "10.0.3.0/24"
  },
  {
    "ParameterKey": "PrivSubnet02CIDR",
    "ParameterValue": "10.0.0.0/24"
  }
]
```

C. Implementation Phase: Network Infrastructure

This study covers the resources that were created for our stack and the specifications assigned to them; for more info regarding resources and their specifications, please visit: [12]

1) **VPC:** Once the parameters have been defined, we move on to the implementation of the network infrastructure of the solution. The first step in the creation of the infrastructure is the creation of a Virtual Private Cloud (VPC), which acts as the container that will include all of the other resources while allowing them to interface and communicate with each other.

```
MainVPC:
  Type: AWS::EC2::VPC
  Properties:
    CidrBlock: !Ref VpcNetCIDR
    EnableDnsSupport: true
    EnableDnsHostnames: true
  Tags:
    - Key: "Name"
      Value: "MainVPC"
```

2) *Subnets*: The next step would be the creation of the subnets, which will be spread across two availability zones, one public and one private subnets per zone; this allows us to prevent the more critical resources from interfacing directly with the internet through assigning them to the private subnets, which can only communicate with the public subnets.

```
PublicSubnet1:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref MainVPC
    AvailabilityZone: "The chosen
      availability zone"
    CidrBlock: !Ref PubSubnet01CIDR
    MapPublicIpOnLaunch: true
    Tags:
      - Key: 'Name'
        Value: 'PublicSubnet1'
```

3) *Route Tables*: With the subnets created, the next step would be defining how the packets and data are route throughout the network and across the subnets through the usage of the route table resources. One route table was created for both public subnets, while a unique table was created for each of the private subnets.

```
PubRouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref MainVPC
    Tags:
      - Key: "Name"
        Value: "PubRouteTable"
```

4) *Gateways*: The following step would be the creation of the gateways: two network address translation gateways (NAT gateways), which will allow the private subnets to interface publicly with the public subnet, and one internet gateway (IGW), which will act as a sign and address for our network to allow it to have outside connections; additionally, in order to allow the NAT gateways to be visible and have a way of connecting to them, an elastic IP (EIP) was assigned to each of the NAT gateways.

```
InternetGW:
  Type: AWS::EC2::InternetGateway
  Properties:
    Tags:
      - Key: "Name"
        Value: "InternetGW"
```

```
NatGw01:
  Type: AWS::EC2::NatGateway
  Properties:
    AllocationId: !GetAtt
      NatGwEIP01.AllocationId
    SubnetId: !Ref PublicSubnet1
    Tags:
      - Key: "Name"
        Value: "NatGw01"
```

```
NatGwEIP01:
  Type: AWS::EC2::EIP
  Properties:
    Tags:
      - Key: "Name"
        Value: "NatGwEIP01"
  DependsOn: IGWAttach
```

D. Implementation Phase: Auto-scaling Group

Once the network infrastructure has been implemented, we may look towards adding more services to the solution to increase its functionality and stability; for instance, since we are hosting a website on the network infrastructure through using elastic cloud compute instances (EC2 instance), which act as servers, it would be beneficial to have a server infrastructure that would be able to scale during times of high traffic so that our service would still be accessible to our users; this can be achieved through the usage of an auto-scaling groups (ASG).

An ASG is a service that allows developers to add policies that would either scale their server infrastructure up through adding more servers to the infrastructure, or scale it down through the termination of unneeded resources. The creation of the instances is done through the usage of a service known as launch configurations, inside of which we may specify the hardware that we want our servers to use as well as any commands that may wish to run during the initialization of the instances; additionally, instances created in through the ASG may be utilized by a resource known as the load balancer (LB) through assigning the instances to a target group (TG). Inside the launch commands in the launch configuration, also known as user data, we specified a GitHub repository from which PHP will be pulled and used to create the website that will be hosted on the instances.

```
ServersAutoScalingGroup:
  Type: AWS::AutoScaling::AutoScalingGroup
  Properties:
    LaunchConfigurationName: !Ref LaunchConfig
    MinSize: '2'
    MaxSize: '3'
    VPCZoneIdentifier:
      - !Ref PublicSubnet1
      - !Ref PublicSubnet2
    TargetGroupARNs:
      - !Ref LBTargetGroup
```

```
LaunchConfig:
  Type: AWS::AutoScaling::LaunchConfiguration
  Properties:
    UserData:
      Fn::Base64: !Sub |
        #!/bin/bash
        yum update -y
        yum install -y httpd
        yum install -y git
        yum install -y php
        yum install -y php-mysql
        amazon-linux-extras install php7.4
        systemctl start httpd
        systemctl enable httpd
```

```

    cd /var/www/html
    curl -o index.php "A link to the raw
      GitHub code"
ImageId: ami-06616b7884ac98cdd
SecurityGroups:
  - !Ref ServerSecGroup
InstanceType: t2.micro
LaunchConfigurationName: HAProjectConfig
BlockDeviceMappings:
  - DeviceName: /dev/sdm
    Ebs:
      VolumeSize: '8'

```

E. Implementation Phase: Load Balancer

LBs are a crucial resource that any developer who wishes to use an instance as a web server should utilize; assume that you have two instances running simultaneously, if an LB isn't implemented, users will direct all of their traffic to the instance to which they were connected to initially, and if the URL associated with one instance is used more often than the other, then the more popular one might become overloaded, while the other instance's resources are being wasted.

The aforementioned situation is the reason behind the creation of the LB; instead of sharing the URL associated with each instance individually, a developer will instead share the link associated with the LB, which would cause the LB to then direct packets and traffic in a manner that would maintain the balance of the resource usage across the instances connected to it.

In order for the LB to know which instances it should direct traffic to, a TG is used; TGs are a group of instances that may be assigned to an LB to inform it of which instances it should balance the load across. Instances created through the ASG can be assigned to a TG automatically, allowing new instances to be created in order to lighten the load across the instances that are already running.

Let's assume another scenario: a developer is running a web server that users may use to perform some task that requires heavy CPU usage, and not all of the users connected may perform that task; should all the users performing the task connect to the same instance, wouldn't that cause one of the instances to overload? Well, another great resource associated with LBs was created, known as the listener; the listener, as the name suggests, listens and monitors resource consumption across a TG and, accordingly, direct traffic across its instances; listeners use a listener rule to indicate on which basis should the load balancing occur; so for the earlier example, our developer might opt to use a listener rule that would balance the load according to CPU usage instead of network traffic. Adding a listener rule requires a developer to have good knowledge of the services that they would offer.

```

LB:
  Type:
    AWS::ElasticLoadBalancingV2::LoadBalancer
  Properties:
    Subnets:
      - !Ref PublicSubnet1

```

```

    - !Ref PublicSubnet2
SecurityGroups:
  - !Ref LBSecGroup

```

```

LBTargetGroup:
  Type:
    AWS::ElasticLoadBalancingV2::TargetGroup
  Properties:
    HealthCheckIntervalSeconds: 10
    HealthCheckPath: /
    HealthCheckProtocol: HTTP
    HealthCheckTimeoutSeconds: 8
    HealthyThresholdCount: 2
    Port: 80
    Protocol: HTTP
    UnhealthyThresholdCount: 5
    VpcId: !Ref MainVPC

```

```

LBListenerRule:
  Type:
    AWS::ElasticLoadBalancingV2::ListenerRule
  Properties:
    Actions:
      - Type: forward
        TargetGroupArn: !Ref LBTargetGroup
    Conditions:
      - Field: path-pattern
        Values: [/]
    ListenerArn: !Ref LBListener
    Priority: 1

```

F. Implementation Phase: Relational Database Service

Last but not least is the relational database service (RDS); RDS is a service offered by AWS that allows developers to create database instance (DB instances), which may then be used as web servers that exclusively host databases on the cloud.

Most enterprises need a database to host their data and their users' data, and instead of hosting the database on a physical server, we decided to host the database on the cloud alongside the rest of the solution and the resources; this allows us to monitor all of our resources and manage them through the cloud.

While some developers can be turned off by letting another party host their database for them, since they would prefer to overlook and manage its security themselves, AWS does offer strong security measure and multiple levels of authentication to prevent data breaches.

```

database1:
  Type: AWS::RDS::DBInstance
  Properties:
    AllocatedStorage: 20
    AllowMajorVersionUpgrade: true
    AutoMinorVersionUpgrade: true
    AvailabilityZone: "The chosen
      availability zone"
    DBInstanceClass: db.t3.micro
    DBInstanceIdentifier: database-1
    DBName: myDB

```

```

DBSecurityGroups:
  - !Ref DBSecGroup
DBSubnetGroupName: !Ref PublicSubnet1
Engine: MySQL
MasterUsername: ''
ManageMasterUserPassword: true
MasterUserPassword: ''
MaxAllocatedStorage: 22
NetworkType: 'IPv4'
Port: '3306'
StorageEncrypted: true
StorageType: 'gp2'
VPCSecurityGroups:
  - !Ref DBSecGroup

```

IV. SOLUTION TESTING PHASES

A. Testing: Creation

With both the parameter file and the infrastructure files created and ready to be deployed, the next step would be actually uploading an deploying them; to do so, a virtual machine (VM) using Linux as an operating system (OS) was used; a developer may opt to use either the git extension available in some IDEs or uploading the files directly to AWS's CloudFormation service instead; all options are viable and it is simply a matter of preference. Once the command has been executed, the files start uploading and the resources start getting created.

```

aws cloudformation create-stack --stackname
  "Stack Name" --region "Region Chosen"
  --template-body "The YAML file"
  --parameters "The parameters file"

```

B. Testing: Updating and Deleting

An attractive aspect of using IaC is the ability to update and delete stacks, which are the resources created through uploading IaC, without having to go over every resource manually; a developer may instead simply edit the values to whatever they need in another file and proceed with executing an update command that would then upload the files once more. To test this feature, we uploaded one faulty file that would cause an error, then we changed the values of the availability zones in some resources in the second attempt.

```

aws cloudformation update-stack --stack-name
  "Stack Name" --region "Region Chosen"
  --template-body "The updated YAML file"

```

Another attractive aspect of using IaC is the ability to delete the stack with the execution of a single command, or through the click of a button on the CloudFormation page, allowing the developer to get rid of all the resources created immediately.

```

aws cloudformation delete-stack --stack-name
  "Stack Name"

```

V. RESULTS

A. Results: Infrastructure

After the upload process was complete, the CloudFormation service created the VPC and assigned the values specified in the YAML file to it.

AWS::EC2::VPC  CREATE_COMPLETE

Fig. 3. The log showing the creation of the VPC.

Once the VPC was created, CloudFormation started the creation of the two sets of two subnets that were specified earlier.

AWS::EC2::Subnet  CREATE_COMPLETE

Fig. 4. The log showing the creation of one of the subnets.

The next resource to be created is the IGW, which finally allows the VPC to interface with the internet; the IGW was provided with the ID of the VPC to specify what is it going to act as a gateway for.

AWS::EC2::InternetGateway
ay  CREATE_COMPLETE

Fig. 5. The log showing the creation of the IGW.

The NAT Gateways and the EIPs were then created, allowing the private subnets to have a secure and private method to interface with the internet.

AWS::EC2::NatGateway  CREATE_COMPLETE

Fig. 6. The log showing the creation of one of the NAT Gateways.

AWS::EC2::EIP  CREATE_COMPLETE

Fig. 7. The log showing the creation of one of the EIPs.

The next resources to be created are the route tables; with their creation, the subnets can start communicating with each other. The two public subnets are sharing the same route table, while each private subnet is having a unique route table.

AWS::EC2::RouteTable  CREATE_COMPLETE

Fig. 8. The log showing the creation of one of the Route Tables.

B. Results: ASG

Once the network infrastructure's creation is finalized, the CloudFormation service moves on to the additional services that were specified, and the ASG and the launch configuration were both created according to the specification; additionally, instances were created since to match the minimum number of instances that was specified to the ASG.

AWS::AutoScaling::AutoScalingGroup  CREATE_COMPLETE

Fig. 9. The log showing the creation of the ASG.

AWS::AutoScaling::LaunchConfiguration  CREATE_COMPLETE

Fig. 10. The log showing the creation of the ASG's Launch Configuration.

C. Results: LB

Then the LB and the TG were created, allowing the load coming to the two instances to be split among them. When connecting to the instances through the URL associated with the LB, the requests would alternate between the instances created, showing that the LB is distributing the load coming to it across the instances in the TG.

AWS::ElasticLoadBalancingV2::LoadBalancer  CREATE_COMPLETE

Fig. 11. The log showing the creation of the LB.

AWS::ElasticLoadBalancingV2::TargetGroup  CREATE_COMPLETE

Fig. 12. The log showing the creation of the TG.

AWS::ElasticLoadBalancingV2::Listener  CREATE_COMPLETE

Fig. 13. The log showing the creation of the Listener.

D. Results: RDS

The RDS database instance was created according to the specification provided in the YAML file; as mentioned earlier, we created a database inside the instance, allowing us to immediately start storing and retrieving data. A developer may use a website to either interact with the database, or, alternatively, use some third party tools to connect to it.

AWS::RDS::DBInstance  CREATE_COMPLETE

Fig. 14. The log showing the creation of the DB Instance.

E. Results: Testing

After executing the creation command, the resources started being created and after around 3-4 minutes, the resources were ready to be used with a website hosted on the EC2 instances,

allowing us to immediately access the database created, adding and viewing records into it through the pulled PHP files.

Once the creation of the resources is finalized, the update command was executed twice; in the first instance, the update didn't go through and was rolled back preventing the solution from getting corrupted and failing through integrating the faulty code into the solution.

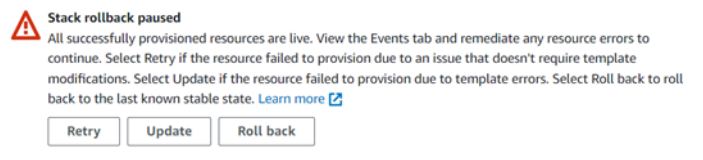


Fig. 15. An error message received upon uploading the faulty file.

In the second instance, the update was deemed compatible with the rest of the solution and was successfully integrated into the solution.

myStack01  UPDATE_IN_PROGRESS

Fig. 16. The log showing that the update compiled successfully.

Finally, the deletion command was executed to delete the entire stack and the resources created; within 2 minutes, the stack and all traces of it were removed.

myStack01  DELETE_IN_PROGRESS

Fig. 17. The log showing that the deletion is in progress.

VI. CONCLUSION AND FUTURE WORK

A. Conclusion

After running this study, it was concluded that IaC can indeed create an enterprise ready infrastructure as well as host a website through which the enterprise can access a database; the process of the creation of the solution through AWS's CloudFormation service was discussed thoroughly: the infrastructure was created and the role of each resource in it was explained, the additional services were specified and implemented, and the additional tests showed how a developer may perform other tasks through the usage of the CloudFormation service.

Once the upload phase was complete, the study showed the resulting solution and the website that we were able to host on the infrastructure; the infrastructure's resources were shown after their creation, whose specifications matched the ones specified in the parameter JSON and the deployment YAML files; then the additional resources were discussed, how their specifications matched the files and how each of them functioned; then we went over the additional tests that were concerned with the functionalities that CloudFormation

provides: the stack was firstly updated successfully, then we tried to update it using a file that didn't compile causing a roll-back, and finally, we deleted the entire stack and the resources associated with it.

B. Future Work

The next step to take with IaC would be the streamlining of the process, which is the step that AWS's CloudFormation is on the track of; for instance, as discussed earlier, users can create, update, or delete the stack through the CloudFormation portal; additionally, users may import other resources into the stack, allowing them to manage their resources all in one place, without having to create another stack and doing a lot of micro-managing, and there is room for more advancements regarding efficiency and the order by which the resources are created, which may be optimized further.

While CloudFormation does allow users to create resources using a graphical designer, it is still too complicated for its own good and requires the users to have a background in network infrastructure and the resources that they want to create; a proposed solution would be having a Graphical User Interface (GUI) that carries users through the process, allowing them to easily specify what they want.

REFERENCES

- [1] Abualkashik, Abedallah and Alwan, Ali and Gulzar, Yonis, "Disaster Recovery in Cloud Computing Systems: An Overview. International Journal of Advanced Computer Science and Applications". 11. 702. 10.14569/IJACSA.2020.0110984, 2020.
- [2] E. Griffith, "What is Cloud Computing?", PC Magazine, Feb. 15, 2022, Available: <https://www.pcmag.com/how-to/what-is-cloud-computing>, [Accessed May 02, 2023].
- [3] Sheth, S. Bhosale, Kadam, "Research Paper on Cloud Computing", Contemporary Research In India (ISSN 2231-2137): Special Issue : April, 2021.
- [4] M. Artać, T. Borovssak, E. Nitto, M. Guerriero, D. Tamburri, 'DevOps: Introducing Infrastructure-as-Code', 497-498. 10.1109/ICSE-C.2017.162, 2017.
- [5] J. Sandobalín, E. Insfran and S. Abrahão, "On the Effectiveness of Tools to Support Infrastructure as Code: Model-Driven Versus Code-Centric", in IEEE Access, vol. 8, pp. 17734-17761, doi: 10.1109/ACCESS.2020.2966597, 2020.
- [6] G. Falazi et al., "On Unifying the Compliance Management of Applications Based on IaC Automation", 2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C), Honolulu, HI, USA, pp. 226-229, doi: 10.1109/ICSAC54293.2022.00050, 2022.
- [7] CH. Raghavendran, G. Satish, P. Varma, G. Moses, "A Study on Cloud Computing Services," International Journal of Engineering Research & Technology (IJERT), ICACC - 2016 Conference Proceedings, Special Issue – 2016, ISSN: 2278-0181, 2016.
- [8] K. Xiong, & H. Perros, "Service Performance and Analysis in Cloud Computing", SERVICES 2009 - 5th 2009 World Congress on Services. 1. 693-700. 10.1109/SERVICES-I.2009.121, 2009.
- [9] C. BasuMallick, "What Is Infrastructure as Code? Meaning, Working, and Benefits", Spiceworks, August 26, 2022, Available: <https://www.spiceworks.com/tech/cloud/articles/what-is-infrastructure-as-code/>, [Accessed May 03, 2023].
- [10] <https://clients.amazonworkspaces.com/Health.html>
- [11] <https://www.lucidchart.com/pages/>
- [12] <https://docs.aws.amazon.com/cloudformation/index.html>